Development Team Project: Design Document

University of Essex

MsC Computer Science

Group 2

Team Members: Ben Waters, Vedran Skvorc, Moses Ngwerume, Ruben Marques

Table of Contents

Introduction *3*

System Requirements and Assumptions *3*

Overview 3

Functional Requirements 3

Security Requirements 4

Design Decisions *4*

Methodology 4

Following the Solo Scrum Methodology, the project uses short iterative sprints for planning, implementation, and review. This structure is ideal for single-developer management (Mohd Zin et al., 2024). Development strictly follows SOLID principles to ensure a modular, maintainable, and scalable codebase (Yanakiev and Lazar, 2024). 3

Software Design 4

Data Structures 5

API Choice 5

Security Risks and Mitigations *5*

UML Diagrams *1*

Class Diagram 1

Use Case Diagram 1

Misuse Case Diagram (Pauli and Xu, 2005) 2

Misuse Case Mitigation Diagram 2

Tools and Libraries *3*

Testing *3*

Unit Testing 3

Integration Testing 3

Acceptance Testing 3

Data Protection *4*

Appendixes *5*

Appendix 1 - Table of Requirements with prioritisation using MoScoW framework 5

Appendix 2 - GDPR Compliance *8*

References *9*

Introduction

The following design specification details a secure backend infrastructure built for a retail clothing store. Through a text-based Command-Line Interface (CLI), approved accounts can execute core Create, Read, Update, and Delete (CRUD) tasks across stock records, user profiles, and purchases. To maintain strict GDPR compliance while bypassing the complexities of PCI-DSS, financial transactions are entirely simulated. Additionally, the architecture enforces Role-Based Access Control (RBAC) to separate administrator, clerk, and customer permissions. This approach actively reduces the application's attack surface in accordance with secure development guidelines.

System Requirements and Assumptions

Overview

The system requirements have been created from retail domain analysis, user stories and cybersecurity threat consideration.

A detailed prioritisation of system requirements using the MoSCoW framework is provided in *Appendix 1*.

Functional Requirements

- Secure user account creation with credential management (password policy and hashing).
- Role-based authentication for users.
- Multi-factor authentication (MFA) via one-time code during login.
- CRUD operations across users, products, and orders.
- Inventory management and stock monitoring.
- Order placement and payment simulation.
- Product browsing and keyword search functionality.
- Command Line Interface (CLI) access for all system interactions.
- External API integration to simulate extended retail services.

These features represent common functional components of modern digital retail platforms (D'Adamo et al., 2021).

Security Requirements

To minimise the system attack surface, the application will implement:

- Password hashing and secure credential storage.
- Role-based authorisation controls.
- Input validation to prevent injection attacks.
- Simulated TOTP and execution delays to strengthen authentication and mitigate automated attacks.
- Login attempts monitoring and limits to mitigate brute force threats.
- Event logging for security monitoring.
- Encryption of sensitive user data.
- Security capability toggle to enable or disable protections during testing.

These controls align with secure-by-design development practices (Shostack, 2014).

Key Assumptions

- The system is accessible exclusively through a Command Line Interface.
- The system will run in a local environment and will not be publicly hosted.
- The primary purpose of the system is to showcase secure development principles, not to function as a production-ready retail platform.
- Third-party libraries will be used for development.

Design Decisions

Methodology

Following a solo developer approach (Pagotto et al., 2016), the project uses short iterative sprints for planning and implementation. This framework ensures codebase maintainability while mitigating the risks of developing without peer review (Moyo and Mnkandla, 2020).

Software Design

The Model-View-Controller (MVC) pattern describes the architecture of a system of objects, separating concerns across three core components: the model, view, and controller enabling maintainability and scalability within the e-retail system (Ramírez-Noriega et al., 2021).The Strategy

pattern is used to toggle the system's security posture, allowing dynamic selection between a SecurePolicy and an InsecurePolicy to demonstrate vulnerability testing (Meiappane, Prabavadhi, & Prasanna Venkatesan, 2013).

Data Structures

The data repository uses the following data structures (Lafore, Broder and Canning, 2022):

- Dictionary: These will be used for user profiles and product records. This allows lookups for authentication requirements and for querying inventory collections.
- List: These will be used for ordered collections that support sequential processing.
- Set: These will be used for unique collections. Examples of this are session tokens and roles, enforcing authorisation constraints.
- JSON: All structures are serialised to a local json file for stateful storage between CLI sessions.

API Choice

Public Currency API : The system will leverage an external REST API using the Python requests library to fetch real-time exchange rates during the checkout process.

Payment API: Stripe API in Sandbox mode to simulate checkout operations.

All API keys will be stored as environment variables rather than being hardcoded. This is inline with OWASP secure coding guidelines (OWASP, 2023).

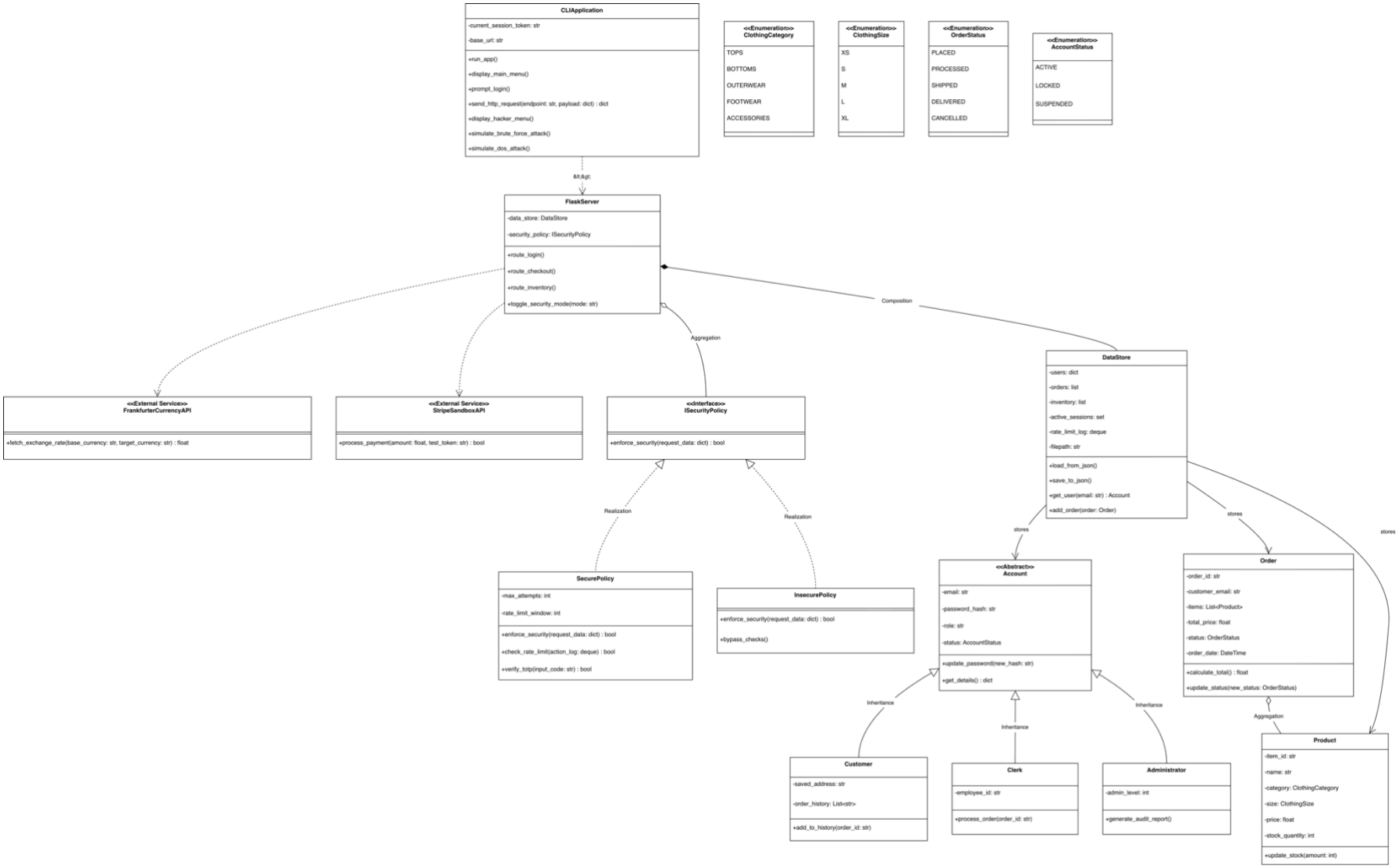
Security Risks and Mitigations

Risk	OWASP	STRIDE	Description	Mitigation
Broken Access Control	A01: Broken Access Control	Elevation of Privilege (E) Tampering (T)	Users may attempt restricted access to functions beyond their assigned role.	Enforce strict Role-Based Access Control (RBAC).
Injection Attacks	A03: Injection A04: Insecure Design	Spoofing (S) Tampering (T) Information Disclosure (I)	Poorly secured APIs or insufficient input validation may allow attackers to	Secure APIs using token-based authentication and enforce strict input validation.

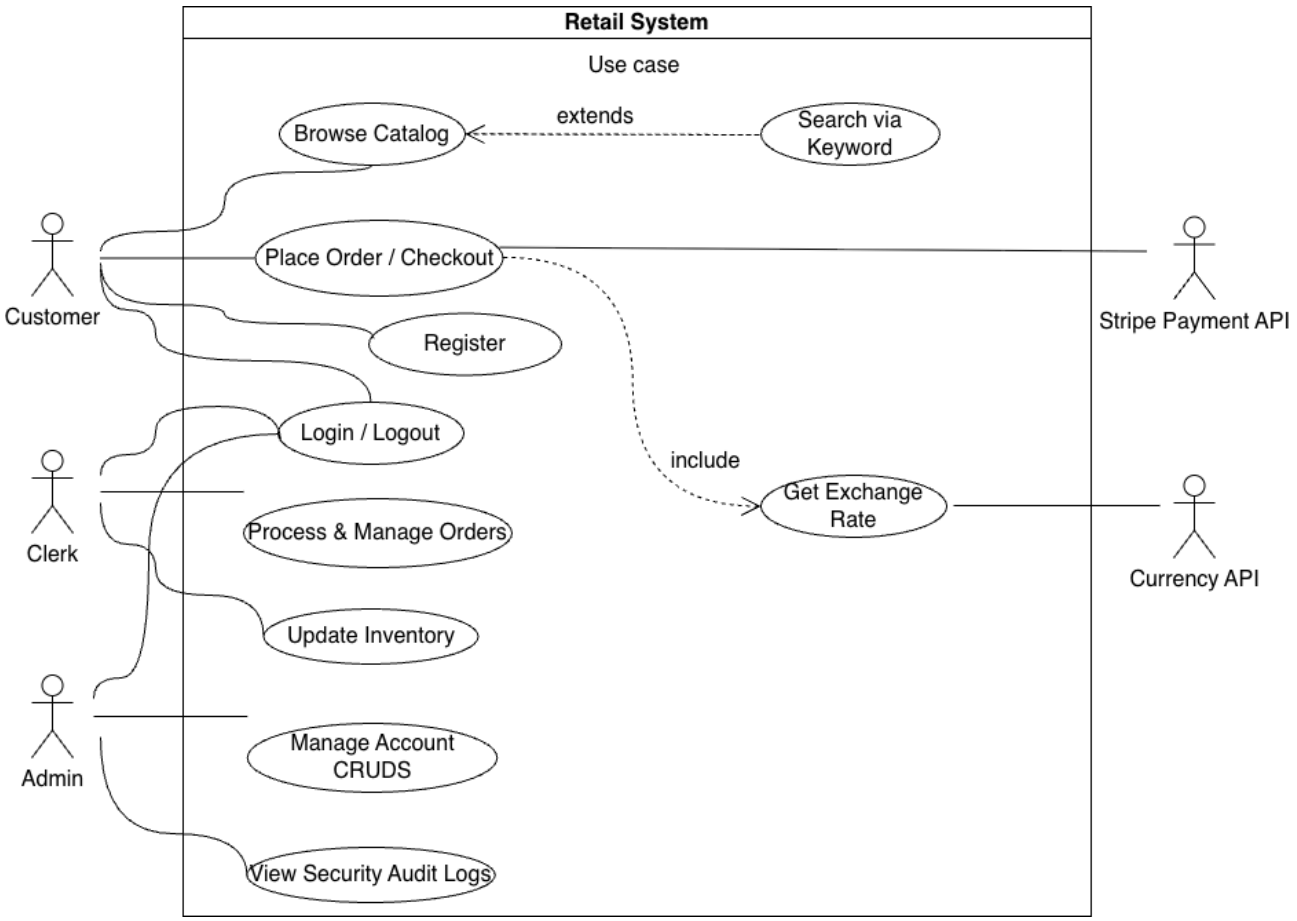
			bypass access controls.	
Denial of Service (DoS) Attacks	A04: Insecure Design A09: Security Logging & Monitoring Failures	Denial of Service (D)	Excessive or malicious traffic may overwhelm system resources, leading to service disruption or downtime.	Apply rate-limiting controls and monitor abnormal traffic patterns.
Brute Force & Credential Attacks	A04: Insecure Design A07: Identification & Authentication Failures A09: Security Logging & Monitoring Failures	Spoofing (S) Repudiation (R) Elevation of Privilege (E)	Attackers may use credential stuffing or password spraying techniques to obtain unauthorised access to user accounts.	Enforce Multi-Factor Authentication (MFA) and implement account lockout after three failed login attempts.
Weak Authentication Mechanisms	A07: Identification & Authentication Failures	Spoofing (S) Repudiation (R) Elevation of Privilege (E)	Inadequate password policies or insecure session handling may enable unauthorised access to sensitive system functions.	Implement strong password requirements, secure session management, MFA and Role-Based Access Control (RBAC).

UML Diagrams

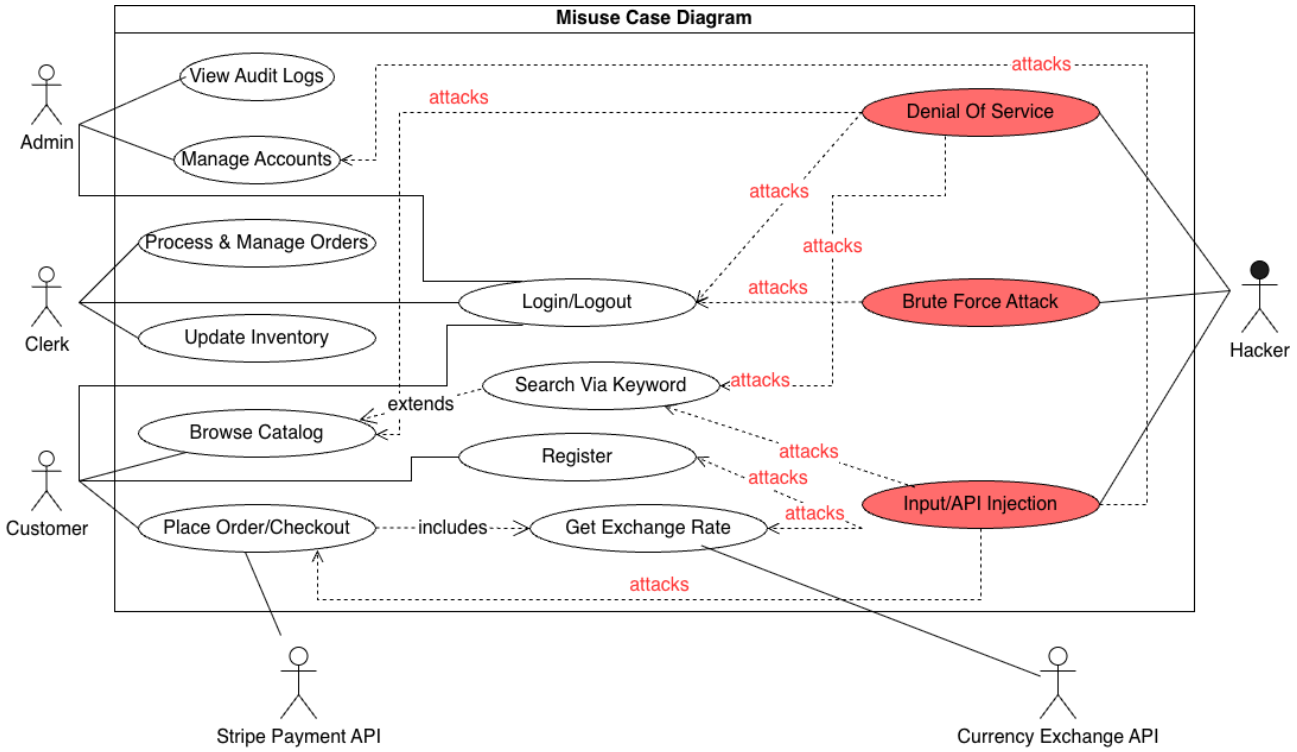
Class Diagram



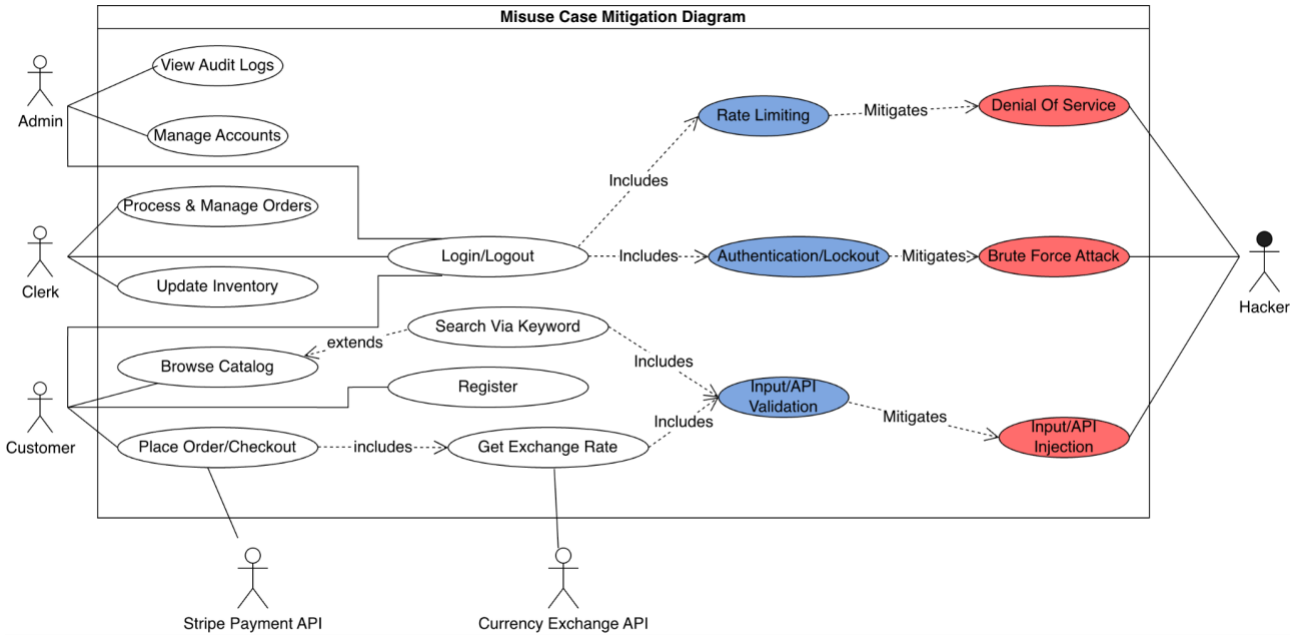
Use Case Diagram



Misuse Case Diagram (Pauli and Xu, 2005)



Misuse Case Mitigation Diagram



Tools and Libraries

Category	Tool / Library	Purpose
Development & Version Control	Python 3	Core programming language
	Git & GitHub	Source control & collaboration
Security	bcrypt	Password hashing
	cryptography	Encryption key management
	hashlib	Data integritypyJWT
	pyJWT	API token authentication
API Handling & Validation	requests	HTTP request handling
	Flask	REST API framework
Input Validation	regex	Custom input validation
Logging & Monitoring	logging	Event tracking

Testing

The Testing strategy will follow a structured design to cover the application requirements. These will include the following test levels.

Unit Testing

Unit tests will be written using Python's unittest framework to target individual components in isolation, ensuring predictable testing of authentication, CRUD operations and input validation (Olmsted, 2024, p.163).

Integration Testing

Integration tests will validate component interactions interactions between components, such as Stripe API connection, role-based access control and session management. This ensures that the integrated components work as expected when combined into a complete system (Olmsted, 2024, p.193).

Acceptance Testing

Acceptance testing will be conducted as confirmation that the system meets the system requirements, determining requirement satisfaction (Olmsted, 2024, p.200).

Data Protection

The application minimises personal data collection to only essential information in line with Article 6(1)(f) of the GDPR (GDPR, 2024a). User data is handled in accordance with GDPR principles to ensure protection of user rights and secure data processing.

Appendixes

Appendix 1 - Table of Requirements with prioritisation using MoScoW framework

Requirement Type	Requirement	Priority	Justification
Cybersecurity & Privacy	Secure User Authentication	Must Have	Ensures only authorised users access the system, protecting customer and operational data from unauthorised entry (Stallings, 2018).
	Password Hashing & Credential Storage	Must Have	Prevents plaintext password exposure and reduces risk in the event of a data breach.
	Multi-Factor Authentication (MFA)	Should Have	Adds an additional verification layer, reducing the likelihood of successful credential compromise attacks.
	Data Encryption	Must Have	Protects sensitive personal information in alignment with GDPR data protection principles (European Parliament, 2016).
	Login Attempt Monitoring	Must Have	Detects and mitigates brute force attacks through lockout or rate-limiting mechanisms.
	Event Logging & Monitoring	Should Have	Enables administrators to audit system activity and detect suspicious behaviour.

	Input Validation Controls	Must Have	Prevents injection attacks targeting system inputs and APIs (OWASP, 2023).
	Security Capability Toggle	Must Have	Allows comparative testing of system resilience with protections enabled or disabled.
User & Access Management	Role-Based Access Control (RBAC)	Must Have	Restricts system privileges according to user roles, enforcing least-privilege access principles.
	Secure Account Creation	Must Have	Enables safe onboarding with password policy enforcement.
	User CRUD Operations	Must Have	Supports administrative management of system users.
Retail Operations	Product Management (CRUD)	Must Have	Enables product catalogue maintenance essential for retail functionality.
	Inventory Monitoring	Must Have	Ensures accurate stock tracking and order fulfilment capability.
	Order Placement & Tracking	Must Have	Facilitates purchase simulation and transaction lifecycle management.

	Product Browsing	Must Have	Allows customers to explore available items before purchase.
	Simulated Payments	Should Have	Use Stripe API to simulate payments via CLI
	Product Search	Could Have	Enhances usability through keyword filtering but is not core to system security objectives.
System Architecture	Command Line Interface (CLI)	Must Have	Required user interaction layer as specified in assignment constraints.
	Python Data Structures Repository	Must Have	Supports coursework requirement to utilise multiple data storage structures.
	External API Integration	Should Have	Demonstrates system extensibility and real-world interoperability.
Out of Scope	Web-Based Graphical Interface	Won’t Have	System interaction is restricted to CLI as per design constraints.
	Enterprise Cloud Deployment	Won’t Have	Local simulation sufficient for demonstrating system security controls.

Appendix 2 - GDPR Compliance

GDPR Standard	Implementation in the System
Article 5 – Principles relating to processing of personal data	The system collects only essential customer information required for retail operations, such as account details and order information. Personal data is processed lawfully, stored securely, and used only for its intended purpose.
Article 7 – Conditions for Consent	Users are informed about data collection practices and provide consent when creating an account. Users can withdraw consent or request account removal if they no longer wish their data to be stored.
Articles 12–14 – Transparent Information and Communication	The system provides clear information regarding how personal data is collected, processed, and stored, ensuring transparency in data handling practices.
Articles 15–22 – Rights of the Data Subject	Users are able to access, update, or request deletion of their personal data. The system supports the right to erasure ("right to be forgotten"), allowing users to request removal of their account and associated information.(GDPR, 2024b)

References

D'Adamo, I., González-Sánchez, R., Medina-Salgado, M.S. and Settembre-Blundo, D. (2021) 'E-commerce calls for cyber-security and sustainability: How European citizens look for a trusted online environment', *Sustainability*, 13(12), p. 6752

European Parliament and Council of the European Union (2016) 'Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)', *Official Journal of the European Union*, L 119/1.

GDPR. (2024a) General Data Protection Regulation (GDPR). Available at: https://gdpr.eu/article-6-how-to-process-personal-data-legally/ (Accessed on 5th March 2026).

GDPR. (2024b) General Data Protection Regulation (GDPR). Available at: https://gdpr.eu/article-17-right-to-be-forgotten/ (Accessed on 7th March 2026).

Lafore, R., Broder, A. and Canning, J. (2022) *Data Structures & Algorithms in Python*. Boston: Pearson.

Meiappane, A., Prabavadhi, J. and Prasanna Venkatesan, V. (2013) Strategy pattern: payment pattern for Internet banking. Available at: https://doi.org/10.48550/arXiv.1312.2327 (Accessed on 22nd February 2026).

Mohd Zin, N.A., Mazlan, A.N., Arifin, N., Abdul Rahman, N., Ahmad, M. and Manaf, M. (2024) Applying Personal Scrum Methodology in System Development for Project-Based Learning Course. Available at: https://ir.uitm.edu.my/id/eprint/113381/1/113381.pdf (Accessed on 22 February 2026).

Moyo, S. and Mnkandla, E. (2020) 'A novel lightweight solo software development methodology with optimum security practices', IEEE Access, 8, pp. 33735-33747. Available at: 10.1109/ACCESS.2020.2971000 (Accessed: 7 March 2026).

Olmsted, A. (2024) Security-Driven Software Development. (edition unavailable). Packt Publishing. Available at: https://www.perlego.com/book/4364423/securitydriven-software-development-learn-to-analyze-and-mitigate-risks-in-your-software-projects-pdf (Accessed: 22 February 2026).

OWASP (2023) *OWASP Secure Coding Practices Quick Reference Guide*. Available at: https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/ (Accessed: 9 March 2026).

Pauli J.J and D. Xu, "Misuse case-based design and analysis of secure software architecture," International Conference on Information Technology: Coding and Computing (ITCC'05) - Volume II, Las Vegas, NV, USA, 2005, pp. 398-403 Vol. 2, Available at: https://doi.org/10.1109/ITCC.2005.199 (Accessed: 24th February 2026).

Pagotto, T., Fabri, J.A., Goncalves, J.A. and L'Erario, A. (2016) 'Scrum solo: Software process for individual development', 2016 11th Iberian Conference on Information Systems and Technologies (CISTI), pp. 1-6. Available at: 10.1109/CISTI.2016.7521555

Ramírez-Noriega, A., Martínez-Ramírez, Y., Jiménez, S., Soto-Vega, J. and Figueroa-Pérez, J.F. (2021) inDev: A software to generate an MVC architecture based on the ER model. Available at: https://doi.org/10.1002/cae.22455 (Accessed on 22nd February 2026).

Shostack, A. (2014) *Threat Modeling: Designing for Security*. Indianapolis: John Wiley & Sons.

Stallings, W. (2018) *Cryptography and Network Security: Principles and Practice*. 7th edn. London: Pearson.

Yanakiev, I. and Lazar, B.M. (2024) Applying SOLID Principles for the Refactoring of Legacy Code: An Experience Report. Available at: https://doi.org/10.1016/j.jss.2024.112254 (Accessed on 22nd February 2026).